

Vitesse de la coupole

Explication et protocole terrain — v6.16.0

Observatoire Ubik — DriftApp Web

Destinataire : Serge, sur site

Durée : environ 1 h, dont ~20 min coupole en mouvement

20 septembre 2026

Sommaire

La coupole tourne à $\sim 43^\circ/\text{min}$ alors que le boîtier de commande du constructeur atteint $90^\circ/\text{min}$ avec le même moteur. La cause vient d'être identifiée : ce n'était pas une limite du driver, mais un coût fixe de $121\ \mu\text{s}$ par pas dans l'ancien programme, qui rendait la vitesse du constructeur arithmétiquement inatteignable. Ce document explique le problème, décrit ce qui a été corrigé, et détaille le protocole à dérouler sur place.

1. Ce qui se passait, en deux mots

Depuis le début, la coupole tourne à environ 43°/min alors que le boîtier de commande du constructeur atteint 90°/min avec le même moteur. Tous les essais pour combler cet écart ont échoué, et on avait fini par conclure que le driver était en cause.

C'était faux. Voici l'image qui explique tout.

Pour faire tourner la coupole, le logiciel envoie des impulsions au moteur : une impulsion = un pas. Plus les impulsions sont rapprochées, plus la coupole va vite. On règle donc un délai entre deux impulsions, en microsecondes.

Le problème : l'ancien programme mettait 121 microsecondes à préparer chaque impulsion, avant même de commencer à attendre le délai demandé. C'est comme un facteur à qui on dit « distribue une lettre toutes les 2 minutes » alors qu'il met déjà 2 minutes à marcher jusqu'à la boîte aux lettres : quoi qu'on lui demande, il ne fera jamais mieux qu'une lettre toutes les 4 minutes.

Résultat concret : quand on demandait 120 μ s, le moteur recevait en réalité 247 μ s. D'où les 45°/min mesurés, au lieu des 90 attendus. On mesurait le facteur, pas le moteur.

Et pour atteindre les 90°/min du constructeur, il faudrait 124 μ s entre deux impulsions. L'ancien programme en consommait 121 rien que pour lui. Il restait 3 μ s : c'était arithmétiquement impossible, aucun réglage n'y serait arrivé.

Les mesures de décembre 2025, relues

Les dix mesures faites sur place le 12/12/2025 le montrent sans ambiguïté. Converties en microsecondes réellement écoulées entre deux pas, elles font apparaître un surcoût constant, quelle que soit la vitesse demandée :

Délai demandé	Vitesse mesurée	Délai réel	Surcoût
2000 μ s	5,3 °/min	2087 μ s	+87 μ s
1100 μ s	9,0 °/min	1242 μ s	+142 μ s
550 μ s	16,9 °/min	658 μ s	+108 μ s
300 μ s	25,4 °/min	439 μ s	+139 μ s
150 μ s	38,7 °/min	288 μ s	+138 μ s
120 μs	45,0 °/min	247 μs	+127 μs
110 μ s	« strengstens verboden ! »	—	—

Le surcoût ne bouge pas : 121 μ s en moyenne, sur une plage de délais de 17×. C'est la signature d'un coût fixe, pas d'une limite mécanique.

Quant à la ligne « strengstens verboden », ce n'est pas le moteur qui décrochait : c'est le moment où le délai demandé passait sous le coût du programme. À partir de là, plus rien ne régulait les impulsions, elles partaient n'importe comment, et le moteur se mettait à vibrer. Un problème de régularité, pas de vitesse.

Enfin, l'écart « 42 vs 48 °/min » qui intriguait n'a jamais existé : c'était la même vitesse, calculée avec deux règles de conversion différentes.

Conclusion : le moteur n'a jamais été poussé au-delà de 247 μ s par pas, dans aucune des deux époques du programme. La limite du driver n'a jamais été mesurée — elle a été supposée.

2. Ce qui a été corrigé

Quoi	En clair
La vitesse sort du code	Elle se règle depuis la page Configuration > Avancé, sans SSH. Valeur par défaut : 260 μ s, soit exactement la vitesse actuelle.
Le firmware du Pico	La phase d'accélération calculait une formule compliquée à chaque pas : c'était le dernier endroit où le « facteur » existait encore. Les valeurs sont désormais préparées à l'avance.
Un instrument de mesure	Un script compare, à chaque palier de vitesse, ce qu'on demande et ce que la coupole fait réellement (lu sur l'encodeur). C'est précisément ce qui n'avait jamais été fait.

Tant que le Pico n'est pas flashé et que la vitesse reste à 260 μ s, tout se comporte exactement comme avant. Cela a été vérifié : les commandes envoyées au Pico sont identiques à l'octet près.

3. Ce qu'on ne sait pas encore

La vraie limite du moteur est inconnue. C'est tout l'objet de ce protocole.

Un seul point physique peut encore bloquer : la tension d'alimentation du driver. À 124 μ s par pas, le moteur tourne à environ 600 tr/min, et à cette vitesse le couple dépend directement de la tension. Le boîtier UPAN, qui atteint 96°/min, alimente son driver en 36 V. Si le nôtre est en 24 V, c'est là que ça butera — et ça se corrigera par l'alimentation, pas par le logiciel.

D'où l'étape 2 ci-dessous.

4. Protocole

RÈGLES À RESPECTER

1. Ne pas modifier la vitesse (delay_us) avant d'avoir flashé le Pico (étape 6). L'ancien firmware ne sait pas accélérer jusqu'à une vitesse élevée : le moteur décrocherait.
2. Ne pas lancer le script de mesure avant le flash, pour la même raison.
3. Pendant les mesures, rester près de la coupole et écouter : un décrochage s'entend (bruit de calage, comme lors de l'incident de septembre).
4. Sur le Pico : une seule source d'alimentation USB à la fois.
5. La mise à jour redémarre les services, donc la coupole bouge (calibration au démarrage). Choisir un moment tranquille.

Bloc A — Relevés, sans rien lancer ni démonter

Étape 1 — Relever le modèle exact du driver

La documentation du projet hésite entre deux modèles. Il faut trancher.

- **À faire** : lire l'étiquette collée sur le boîtier du driver moteur (celui qui reçoit les fils du moteur).
- **À rapporter** : le modèle exact (DM556T ou DM860T, ou autre) et, si elle est lisible, la plage de tension indiquée.

Étape 2 — Mesurer la tension d'alimentation du driver

Multimètre en tension continue (DC), calibre 200 V si le réglage est manuel.

- **À faire** : mesurer aux bornes d'alimentation puissance du driver, repérées VDC et GND (ou V+ / V-).
- **Attention** : ne pas confondre avec les bornes PUL+ / DIR+, qui sont les signaux logiques et n'ont rien à voir.
- **Attendu** : une valeur entre 24 et 48 V.
- **À rapporter** : la valeur lue.

Étape 3 — Noter les micro-interrupteurs (DIP)

- **À faire** : photographier ou noter la position des petits interrupteurs sur le côté du driver (ON/OFF, généralement 8).
- **À rapporter** : la photo, ou la suite ON/OFF.

Cela permet de confirmer le réglage de courant et le microstepping (on attend 800 impulsions par tour moteur).

Les étapes 1 à 3 se font sans rien démonter ni rien lancer. Si la tension relevée est basse (24 V), prévenir avant d'aller plus loin : l'objectif de vitesse devra peut-être être revu à la baisse.

Bloc B — Mise à jour, flash et mesures

Étape 4 — Installer la mise à jour 6.16.0

- **À faire** : sur le tableau de bord, cliquer sur « Mettre à jour » et laisser faire.
- **Attendu** : la version affichée en bas de page passe à 6.16.0. La coupole effectue sa calibration habituelle au redémarrage.

Si une bannière de configuration apparaît avec la mention « migrated » : c'est normal, elle signale simplement qu'un nouveau réglage a été ajouté. Aucune valeur du site n'a été touchée.

Étape 5 — Vérifier que rien n'a changé

- **À faire** : utiliser la coupole normalement — un GOTO, quelques JOG, éventuellement un début de suivi.
- **Attendu** : strictement le même comportement qu'avant, même vitesse, même son.

C'est la vérification la plus importante du protocole : si quelque chose diffère ici, s'arrêter et prévenir.

Étape 6 — Sauvegarder puis flasher le Pico

En SSH sur le Pi :

```
ssh slenk@<pi-du-site>
cd ~/DriftApp

# 1. SAUVEGARDE de l'ancien firmware – a ne pas sauter
mkdir -p ~/firmware_backup
mpremote cp :main.py          ~/firmware_backup/main.py
mpremote cp :step_generator.py ~/firmware_backup/step_generator.py
mpremote cp :ramp.py           ~/firmware_backup/ramp.py
ls -l ~/firmware_backup      # doit lister les 3 fichiers

# 2. Flash du nouveau firmware
cd ~/DriftApp/firmware
mpremote cp main.py          :main.py
mpremote cp step_generator.py :step_generator.py
mpremote cp ramp.py           :ramp.py
```

- **Attendu** : aucune erreur. Le Pico redémarre tout seul.
- **Retour arrière** : à tout moment, recopier les trois fichiers de ~/firmware_backup vers le Pico (mêmes commandes dans l'autre sens) et débrancher/rebrancher le Pico.

Étape 7 — Re-vérifier que rien n'a changé

- **À faire** : redémarrer les services (page Configuration > « Redémarrer les services »), puis refaire un GOTO et quelques JOG.
- **Attendu** : là encore, comportement identique. Le flash est censé être neutre tant que la vitesse reste à 260 µs.

Si ce n'est pas le cas, revenir à l'ancien firmware (étape 6) et prévenir.

Étape 8 — Campagne de mesure

C'est l'étape où la coupole va tourner de plus en plus vite. Rester à proximité.

```
cd ~/DriftApp

# D'abord voir le plan, sans rien faire bouger
python3 scripts/diagnostics/calibration_vitesse_encodeur.py --dry-run

# Puis la mesure réelle (demande de taper « go » avant de démarrer)
python3 scripts/diagnostics/calibration_vitesse_encodeur.py
```

Le script teste dix vitesses, de la plus lente à la plus rapide. À chaque palier il fait tourner la coupole de 10° et compare la vitesse obtenue à la vitesse demandée. Il affiche un tableau de ce type :

délai	mesuré	visé	rendement
260 µs	42.8°/min	42.8°/min	100 %
235 µs	47.2°/min	47.3°/min	100 %
...			

- **Arrêt automatique** : le script s'arrête seul au premier palier où la coupole ne suit plus, et annonce une valeur recommandée.
- **Arrêt manuel** : Ctrl-C interrompt tout immédiatement si quelque chose semble anormal.
- **À rapporter** : le tableau complet affiché en fin de script, et le fichier logs/calibration_vitesse_*.json.

Étape 9 — Régler la vitesse définitive

- **À faire** : page Configuration > Avancé > motor_driver > delay_us, saisir la valeur recommandée par le script (jamais une valeur plus basse), enregistrer, puis « Redémarrer les services ».
- **Attendu** : la coupole est plus rapide, sans bruit anormal.

À valider sur une vraie session avant de considérer l'affaire close : un suivi complet, avec un passage au méridien si possible.

5. Résumé de ce qu'il faut rapporter

Étape	Information attendue
1	Modèle exact du driver
2	Tension d'alimentation mesurée
3	Position des DIP, ou photo
5 et 7	Confirmation que rien n'a changé (après mise à jour, puis après flash)
8	Tableau de mesures complet + fichier logs/calibration_vitesse_*.json

En cas de doute à n'importe quelle étape : s'arrêter et demander. Rien dans ce protocole n'est urgent, et l'ancien fonctionnement reste disponible à tout moment (retour arrière du firmware + delay_us remis à 260).